Implementation of Unification and Resolution Algorithm

```
import itertools


# Variables: lowercase strings (x, y). Constants: Capitalised (Marcus).
# Compound term: tuple ('f', arg1, arg2...).  Literal: (sign, predicate, args_tuple)
def is_var(t):
    return isinstance(t, str) and t[:1].islower()


def substitute(t, theta):
    if is_var(t):
        return substitute(theta[t], theta) if t in theta else t
    if isinstance(t, tuple):
        return tuple(substitute(a, theta) for a in t)
    return t


def occurs(v, t, theta):
    t = substitute(t, theta)
    if t == v:
        return True
    return isinstance(t, tuple) and any(occurs(v, a, theta) for a in t)


def unify(x, y, theta=None):
    """Returns most general unifier (dict) or None."""
    if theta is None:
        theta = {}
    x, y = substitute(x, theta), substitute(y, theta)
    if x == y:
        return theta
    if is_var(x):
        return None if occurs(x, y, theta) else {**theta, x: y}
    if is_var(y):
        return None if occurs(y, x, theta) else {**theta, y: x}
    if isinstance(x, tuple) and isinstance(y, tuple) and len(x) == len(y):
        for a, b in zip(x, y):
            theta = unify(a, b, theta)
            if theta is None:
                return None
        return theta
    return None


_counter = itertools.count()


def rename(clause):
    """Standardise variables apart."""
    n = next(_counter)
    m = {}

    def r(t):
        if is_var(t):
            return m.setdefault(t, f"{t}{n}")
        if isinstance(t, tuple):
            return tuple(r(a) for a in t)
        return t

    return frozenset((s, p, r(args)) for s, p, args in clause)


def resolve(c1, c2):
    out = []
    c1, c2 = rename(c1), rename(c2)
    for l1 in c1:
        for l2 in c2:
            if l1[0] != l2[0] and l1[1] == l2[1]:
                theta = unify(l1[2], l2[2])
                if theta is not None:
                    rest = (c1 - {l1}) | (c2 - {l2})
```

Implementation of Unification and Resolution Algorithm

```
                    out.append(frozenset((s, p, substitute(a, theta)) for s, p, a in rest))
    return out


def fmt(clause):
    if not clause:
        return "[] (empty clause)"
    return " v ".join(("~" if s == "-" else "") + p + "(" + ",".join(map(str, a)) + ")"
                      for s, p, a in sorted(clause))


def resolution(kb, query_negated, max_iter=500):
    clauses = [frozenset(c) for c in kb] + [frozenset(query_negated)]
    for c in clauses:
        print("  given:", fmt(c))
    seen = set(clauses)
    for _ in range(max_iter):
        new = []
        for c1, c2 in itertools.combinations(clauses, 2):
            for r in resolve(c1, c2):
                if not r:
                    print("  derived:", fmt(r), " <-- contradiction, so the query is PROVED")
                    return True
                if r not in seen:
                    seen.add(r)
                    new.append(r)
                    print("  derived:", fmt(r))
        if not new:
            return False
        clauses += new
    return False


def program7():
    print("\n=== 7. UNIFICATION AND RESOLUTION ===")
    print("-- Unification --")
    t1 = ("Knows", "John", "x")
    t2 = ("Knows", "y", ("Mother", "y"))
    print(f"unify({t1}, {t2}) = {unify(t1, t2)}")
    print("unify(P(A), P(B))  =", unify(("P", "A"), ("P", "B")))

    print("\n-- Resolution refutation --")
    print("KB: Man(Marcus).  Man(x) -> Mortal(x).   Query: Mortal(Marcus)")
    kb = [
        [("+", "Man", ("Marcus",))],
        [("-", "Man", ("x",)), ("+", "Mortal", ("x",))],
    ]
    negated_query = [("-", "Mortal", ("Marcus",))]
    print("Proved:", resolution(kb, negated_query))


if __name__ == "__main__":
    program7()
```